COMPTIA SECURITY+ SY0-701 | DOMAIN 2 · THREATS, VULNERABILITIES, AND MITIGATIONS (22%)

Security+ 2.3 · SQL Injection

Mestre CyberSec · **Lesson 2.3 — SQL Injection** (5:09) · Assistir Vídeo no YouTube ↗ · Treino Interativo: edusouzamarques.github.io/cybersec

Executive Lesson Summary: Injetar SQL num campo de entrada: 'OR 1=1--' pula o login; DROP TABLE apaga a tabela; UNION SELECT extrai dado. Defesa: PREPARED STATEMENT (parameterized query), input validation, WAF, menor privilégio no banco. Nunca concatenar string pra query.

💡 Key Exam Highlights & Critical Concepts

- SQLi = input injetado no query sem sanitizar
- Defesa nº1: prepared statement / parameterized query
- Defesa nº2: menor privilégio (o banco do user não tem Drop Table)

🎯 Exam Tips & Typical CompTIA Scenarios

- 'input concatenated into a SQL query' = SQL injection
- 'parameterized queries prevent' = defesa do SQLi (prepared statement)

🚨 Beyond the Video: Official Exam Concepts, Protocols & Traps (Section 2.3 — SQL Injection)

Why is this advanced coverage required for SY0-701?: Video lectures provide foundational concepts, but CompTIA tests practical engineering judgment and requires deep familiarity with the following protocols, acronyms, and operational nuances in this domain:

• SQL Injection (SQLi)

An injection attack where untrusted input is concatenated into a SQL query, allowing attackers to alter query logic and access or destroy database data.

⚠ ***Exam Trap:*** *Trigger phrase 'input concatenated into a SQL query' = SQLi. Use '--' or ' OR 1=1' as classic textbook payload examples.*

• Prepared Statement (Parameterized Query) (N/A)

A coding technique where SQL code and user input are sent to the DBMS in distinct parameters, so input is treated as data only and cannot alter query structure.

⚠ ***Exam Trap:*** *Trigger phrase 'parameterized queries prevent' = the #1 defense for SQL injection on the exam. Prefer this over escaping or WAF when both are options.*

• Input Validation (N/A)

Server-side checking of user input against an expected format, type, or range (e.g., allowlist of alphanumeric only) before processing, reducing injection risk.

⚠ ***Exam Trap:*** *Distinguish from sanitization: validation rejects bad input, sanitization cleans it. For SQLi, parameterized query is still preferred even when validation exists.*

• Web Application Firewall (WAF)

A layer-7 firewall that inspects HTTP/S traffic to a web application and blocks known malicious payloads such as SQLi, XSS, and command injection patterns.

⚠ ***Exam Trap:*** *WAF is compensating control, NOT a substitute for parameterized queries. On the exam, the best fix for SQLi is always code-level, not network-level.*

• Principle of Least Privilege (PoLP)

Granting a database user account only the minimum permissions required (e.g., SELECT/INSERT only), so a successful SQLi cannot DROP tables or escalate.

⚠️ ***Exam Trap:*** *Trigger phrase 'database user cannot DROP TABLE' = least privilege mitigated impact. Pair with PoLP for defense-in-depth, not as primary SQLi fix.*

• UNION SELECT Attack (N/A)

A SQLi payload using the UNION operator to append a second SELECT statement, exfiltrating data from other tables in the same database through the vulnerable query's output.

⚠️ ***Exam Trap:*** *If exam describes 'extracting data via UNION' or 'appended SELECT', it is SQLi, not XSS or command injection. Requires equal column count to succeed.*

• DROP TABLE Statement (N/A)

A destructive SQL DDL command often injected via SQLi (e.g., '; DROP TABLE users--') to delete entire tables and cause denial of service or data loss.

⚠️ ***Exam Trap:*** *Trigger phrase 'database user should not have permission to DROP' = least privilege mitigation. Multi-statement execution must be enabled for this payload to work.*

• String Concatenation Vulnerability (N/A)

The insecure code pattern of building SQL by joining strings (e.g., 'SELECT * FROM users WHERE name="' + input + '"'), which lets attackers break out of the data context into code.

⚠️ ***Exam Trap:*** *Any exam question showing query built via '+' or string concatenation is the hallmark vulnerable pattern — fix is parameterized query, not escaping.*

🏛️ Conceptual Deep Dive, Real-World Analogies & GRC Analysis

📌 SQL Injection & Parameterized Queries (Prepared Statements)

🏠 **Real-World Analogy:** The waiter and the plastic-slotted order card. In classical SQLi, a diner writes on a napkin: 'Cheeseburger; AND give me all the cash in the register!'. If the waiter shouts this verbatim to the kitchen as restaurant orders, the cook hands over the money. With Parameterized Queries ('?'), the order pad has rigid pre-molded slots labeled 'Burger Flavor': the diner's text can only sit inside that slot. Even if the diner writes 'give me all the cash', the cook simply reads: 'Ah, the guest wants a sandwich whose flavor name is literally give me all the cash'. It is treated strictly as data, never as an executive command.

🔍 **Root Cause & Background:** String concatenation in dynamic SQL statements where instructions and user-supplied data share the same execution channel without syntactic separation, allowing quotes and escape operators to rewrite query logic (e.g., ' OR '1'='1).

🛡️ **GRC Auditor Perspective:** Mandate secure coding policies (SDLC) with SAST rules that fail builds containing raw SQL concatenation. Compliance audits against PCI-DSS Requirement 6.5.1 and CIS Control 16.11, verifying mandatory use of parameterized queries with bind variables across all repositories touching cardholder data or credentials.

⚠️ **How CompTIA Tests This:** CompTIA cues: '1=1', single quotes breaking queries with database syntax errors, or 'attacker bypasses login using ' OR '1'='1 --'. Question: 'What is the PRIMARY defense against SQL Injection?' Answer: Prepared Statements / Parameterized Queries. Distractor trap: Input validation and WAF are secondary depth layers, never the primary architectural fix.

📌 UNION-Based SQLi & Dynamic Stored Procedure Vulnerabilities

🏠 **Real-World Analogy:** The hotel key request and the poisoned procedure manual. In UNION SQLi, a guest asks the front desk: 'Hand me key 101 UNION SELECT all customer SSNs and credit cards'; the naive clerk delivers both on the same tray. An insecure Stored Procedure is like an official leather-bound operations manual whose actual rule states: 'Take whatever random scratch paper the guest hands over and read it directly over the intercom'. The formal manual binding does not prevent the catastrophe.

🔍 **Root Cause & Background:** The UNION operator merges result sets when column counts and data types match. In stored procedures, vulnerability arises when developers invoke dynamic query interpreters (e.g., EXEC(@sql)) rather than typed parameter declarations.

🛡️ **GRC Auditor Perspective:** Database administrator (DBA) schema audits and code scans flagging unparameterized dynamic SQL inside database routines. Enforce least privilege on database service accounts to deny access to system schemas or unauthorized business tables.

⚠️ **How CompTIA Tests This:** CompTIA cues: 'Attacker injects UNION SELECT username, password_hash FROM users'. Prerequisite condition: queries must match identical column numbers and compatible data types. Regarding Stored Procedures: exam tests knowledge that stored procedures DO NOT automatically prevent SQLi if internal strings are dynamically concatenated.

📌 Blind SQL Injection: Boolean-Based & Time-Based

🏠 **Real-World Analogy:** The interrogator probing a silent vault in the dark. The attacker cannot see inside the secure room, but whispers through the intercom: 'If the first letter of the master password is A, wait 10 seconds before turning on the hallway lamp; otherwise turn it on instantly'. Watching the stopwatch, if the lamp takes 10 seconds, he proves that the letter is A. He iterates through each character to extract entire databases.

🔍 **Root Cause & Background:** The application suppresses database error messages (preventing info disclosure), but the underlying SQL engine still evaluates injected conditional logic and heavy time delay primitives (e.g., WAITFOR DELAY, pg_sleep, BENCHMARK).

🛡️ **GRC Auditor Perspective:** Deploy Database Activity Monitoring (DAM) and WAF anomaly detection rules tracking abnormal latency spikes and sleep signatures. Enforce strict database connection query timeouts to abort long-running injected delays.

⚠️ **How CompTIA Tests This:** CompTIA cues: 'Application returns generic error page or uniform response', 'attacker injects WAITFOR DELAY '0:0:10' or sleep(5)', 'data extracted by measuring round-trip network response times'. Answer: Blind SQL Injection (Time-based).

📌 ORM Raw Query Escape Hatch & Input Validation vs. Output Encoding

🏠 **Real-World Analogy:** The airport baggage scanner and the unmonitored service door. An ORM is a high-tech automated baggage scanner that disassembles and inspects everything safely. The 'raw query escape hatch' is the side alley door that a rushed worker leaves open to shove heavy carts through without inspection to save time. Input validation is the guard checking your ticket at the turnstile; output encoding is placing a reinforced glass case over a volatile artifact so spectators can view it without getting touched.

🔍 **Root Cause & Background:** Developers utilizing ORM escape methods such as raw SQL queries with string concatenation to satisfy complex reporting needs. A conceptual confusion between checking input formats versus preventing command execution context.

🛡️ **GRC Auditor Perspective:** Automated code analysis (SAST) flagging any unparameterized ORM raw query invocations as critical defects. Architectural policy mandating strict allowlists for incoming user formats combined with backend parameterization.

⚠️ **How CompTIA Tests This:** CompTIA cues: 'A developer migrated an application to an ORM framework yet a database compromise still occurred' -> Cause: developer utilized raw SQL bypass methods with concatenation. Trap: 'HTML output encoding prevents SQL injection' -> FALSE! Output encoding neutralizes XSS; SQL injection is resolved at the database layer via prepared statements.

⚡ QUICK-REFERENCE EXAM CHEAT SHEET

Typical Exam Scenario / Practical Situation	Technical Concept & Target Answer
Attacker injects ' OR '1'='1' -- and bypasses web application authentication	**Classic SQL Injection (SQLi)**
Primary mandatory architectural defense to eliminate SQL injection across queries	**Parameterized Queries / Prepared Statements with Bind Variables**
Attacker appends records from another table using UNION SELECT	**UNION-Based SQL Injection**
Application suppresses errors; attacker extracts data by measuring response delay	**Time-Based Blind SQL Injection**
Application uses modern ORM but developer uses raw query methods with string concatenation	**Insecure ORM Raw Query Bypass**
Conceptual confusion applying output encoding instead of database query parameterization	**Confusion between XSS Defense (Output Encoding) and SQLi (Prepared Statements)**

✅ Authentic Practice Questions (10 Real Exam Scenarios)

Scenario-based exam questions. Complete explanations and distractor analysis are provided in the answer key below.

QUESTION 1 · EXAM SCENARIO

A retail company's login page authenticates users by building this query: SELECT * FROM Users WHERE username = '<input>' AND password = '<input>'. A penetration tester enters ' OR 1=1-- in the username field and is logged in as the first user in the table. Which change best addresses the root cause of this vulnerability?

A) Replace dynamically concatenated SQL with parameterized prepared statements.

B) Require users to choose longer passwords.

C) Enable account lockout after three failed attempts.

D) Enforce TLS 1.3 for all authentication traffic.

QUESTION 2 · EXAM SCENARIO

During a security assessment, a tester finds that a product search page takes the search term and appends it to a query. By submitting a crafted term, the tester causes the database to return usernames and password hashes in the product description column. Which control most directly prevents this type of data extraction?

A) Deploying a web application firewall in monitor-only mode.

B) Using parameterized queries for the search operation.

C) Hiding database error messages from users.

D) Limiting the search results to ten records per page.

QUESTION 3 · EXAM SCENARIO

A web application connects to its database with an account that has db_owner rights. An attacker exploiting a form input attempts to execute a stacked query that drops the Orders table. Which database-side control would most reduce the impact if the application is successfully injected?

A) Enabling detailed SQL query logging.

B) Storing backups on the same database server.

C) Granting the application account only the permissions required by its functions.

D) Adding a CAPTCHA to the vulnerable form.

QUESTION 4 · EXAM SCENARIO

A code review finds the following Java snippet: String q = "SELECT * FROM accounts WHERE acct = '" + acct + "'"; Which replacement best removes the SQL injection flaw?

A) Replace single quotes in acct with two single quotes before concatenation.

B) Limit acct to ten alphanumeric characters.

C) Move the query to a stored procedure that uses EXEC with concatenated acct.

D) Use a PreparedStatement with a ? placeholder and bind acct as a parameter.

QUESTION 5 · EXAM SCENARIO

A security engineer confirms that a web application firewall is blocking known SQL injection patterns, but the application still concatenates user input into SQL queries. Which additional control most directly reduces the application-layer risk?

A) Refactor database access to use parameterized queries.

B) Segment the web server and database server into different VLANs.

C) Enable file integrity monitoring on the database server.

D) Require multi-factor authentication for database administrators.

QUESTION 6 · EXAM SCENARIO

A developer adds server-side validation that a customer ID field contains only digits before using it in a database query. Which statement best describes the security value of this control?

A) It fully prevents SQL injection and removes the need for parameterized queries.

B) It reduces the attack surface but should be combined with parameterized queries.

C) It is effective only when client-side JavaScript validation is also enabled.

D) It replaces the need for least privilege on the database account.

QUESTION 7 · EXAM SCENARIO

A database administrator discovers that the web application's database account has SELECT, INSERT, UPDATE, DELETE, and DROP privileges on all schemas, even though the application only reads from two views and writes to one table. Which action best reduces the potential impact of a compromised application account?

A) Rename the application database account to a less obvious name.

B) Enable TLS for all database connections.

C) Revoke unnecessary privileges and grant access only to required views and table operations.

D) Move the database to a separate network segment.

QUESTION 8 · EXAM SCENARIO

An application returns raw database errors to users, including table names, column names, and SQL syntax details. An attacker uses these messages to refine SQL injection attempts. Which action best reduces this information disclosure while supporting incident investigation?

A) Disable database query logging entirely.

B) Configure the WAF to only log requests without blocking.

C) Increase the complexity of application user passwords.

D) Return generic error messages to users and log detailed errors on the server.

QUESTION 9 · EXAM SCENARIO

A legacy application cannot be modified immediately to use parameterized queries. The security architect needs a temporary compensating control at the HTTP layer to block malicious SQL injection requests until the code is fixed. Which control best fits this requirement?

A) A web application firewall with SQL injection signatures in blocking mode.

B) Database encryption at rest for all application tables.

C) Role-based access control for application end users.

D) Database credential rotation every 30 days.

QUESTION 10 · EXAM SCENARIO

A development team moves several database operations into stored procedures to improve security. A reviewer finds that one procedure builds a query with EXEC('SELECT * FROM Inventory WHERE Item = ''' + @Item + '''). Which conclusion is most accurate?

A) Stored procedures always eliminate SQL injection regardless of how they are written.

B) The procedure remains vulnerable because input is still concatenated into dynamic SQL.

C) SQL injection only affects ad hoc queries, not stored procedures.

D) The procedure is safe if the database account has least privilege.

ANSWER KEY & COMPREHENSIVE DISTRACTOR EXPLANATIONS

Question 1: Option A — Replace dynamically concatenated SQL with parameterized prepared statements.
Parameterized prepared statements separate SQL code from data, so the input ' OR 1=1-- is treated as a literal username rather than executable SQL. Longer passwords, lockout, and TLS are useful but do not fix the injection flaw.

Question 2: Option B — Using parameterized queries for the search operation.
The described attack is a UNION SELECT-based SQL injection. Parameterized queries prevent user input from being interpreted as SQL syntax, stopping the injected UNION SELECT. A WAF in monitor mode only observes, error hiding does not stop extraction, and result limits may reduce volume but not the flaw.

Question 3: Option C — Granting the application account only the permissions required by its functions.
Least privilege limits what the injected SQL can do. If the application account cannot drop tables or access unrelated objects, a successful injection causes less damage. Logging, same-server backups, and CAPTCHA do not remove excessive database permissions.

Question 4: Option D — Use a PreparedStatement with a ? placeholder and bind acct as a parameter.
A PreparedStatement with parameter binding treats acct as data and prevents it from changing SQL syntax. Manual quote escaping and length limits are incomplete, and a stored procedure using EXEC with concatenation can remain vulnerable.

Question 5: Option A — Refactor database access to use parameterized queries.
Parameterized queries fix the root cause by preventing input from being executed as SQL. Network segmentation, file integrity monitoring, and MFA for DBAs are useful controls but do not directly eliminate application SQL injection vulnerabilities.

Question 6: Option B — It reduces the attack surface but should be combined with parameterized queries.
Input validation can reject many malicious inputs and reduce attack surface, but it is not a complete SQLi defense by itself. Parameterized queries are still required, and least privilege remains important.

Question 7: Option C — Revoke unnecessary privileges and grant access only to required views and table operations.
Least privilege means granting only the minimum permissions required. Revoking broad rights and limiting access to required objects reduces the impact of SQL injection and other compromises. Renaming, TLS, and segmentation do not address excessive database privileges.

Question 8: Option D — Return generic error messages to users and log detailed errors on the server.
Generic user-facing errors avoid revealing schema and query details, while server-side logging preserves diagnostic information for investigators. Disabling logging harms investigation, WAF logging-only does not reduce disclosure, and password complexity does not address error leakage.

Question 9: Option A — A web application firewall with SQL injection signatures in blocking mode.
A WAF with SQLi signatures in blocking mode can provide temporary detection and prevention at the HTTP layer while the legacy code is fixed. Encryption at rest, RBAC for users, and credential rotation do not inspect or block SQL injection payloads.

Question 10: Option B — The procedure remains vulnerable because input is still concatenated into dynamic SQL.
Stored procedures are not automatically safe. If they construct dynamic SQL with concatenated user input and EXEC, injection is still possible. Least privilege reduces impact but does not fix the vulnerable procedure; injection is not limited to ad hoc queries.